

Graphic Simulation

Final Midterm Project

ML Exploration



U N I V E R S I D A D
Panamericana

Institute:

Universidad Panamericana Guadalajara

Team:

Gabriel Torres Zacarias

Cristian R. Miravete Cid

Sebastián Avilez Hernández

Profesor:

Ing. Gabriel Castillo Cortés

Date:

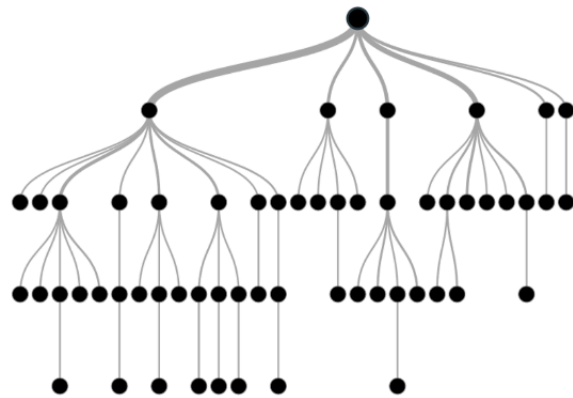
28/05/2025

Index

Algorithm #1: Decision Trees.....	3
Algorithm #2: Support Vector Machines.....	7
Algorithm #3: Random Forest Algorithms.....	10
Conclusions.....	15
Bibliography.....	17

Algorithm #1: Decision Trees

Trees serve as a familiar metaphor in daily life. Composed of roots, trunk, branches, and foliage, trees frequently represent growth and development. In machine learning, decision trees structures earn their name by beginning at the root and extending branches to illustrate different possible outcomes. Since machine learning focuses on problem-solving, decision trees enable us to visualize models and refine our training approaches. Decision trees represent supervised learning algorithms employed for both classification and regression tasks and resemble flowcharts in structure. They begin with a root node containing a specific data query, which leads to branches holding possible responses, these branches connect to internal decision nodes that pose additional questions, generating further outcomes. This process continues until data reaches a terminal node where it concludes.



Machine learning decision trees can function as either classification or regression trees, often abbreviated as CART. Classification trees establish whether an event occurred or not. Typically, this involves binary outcomes like "yes" or "no" responses. We frequently employ this decision-making approach in real-world scenarios. Regression trees, conversely, forecast continuous values using historical data or information sources. For instance, they might predict gasoline prices or customer purchasing behavior regarding eggs. This decision-making style involves programming algorithms to predict likely outcomes based on past behaviors or trends.

Generally, constructing a balanced binary tree requires $O(n_{\text{samples}} * n_{\text{features}} * \log(n_{\text{samples}}))$. While tree construction algorithms attempt creating balanced structures, perfect balance isn't always achieved. Assuming subtrees maintain approximate balance, each node's cost involves searching through $O(n_{\text{features}})$ to identify features offering maximum impurity criterion reduction, such as logarithmic loss. This creates a total tree cost of $O(n_{\text{features}} * n^2_{\text{samples}} * \log(n_{\text{samples}}))$.

Interpretation remains straightforward: beginning at the root node, you progress to subsequent nodes while edges indicate which subsets you're examining. Upon reaching leaf nodes, they provide predicted outcomes. More terminal nodes and increased tree depth make understanding decision rules increasingly challenging. Depth 1 creates 2 terminal nodes. Depth 2 allows a maximum of 4 nodes.

These methods offer versatility and interpretability. Nevertheless, they can become computationally demanding in worst-case scenarios during both training and inference phases. They perform effectively for classification and regression tasks but might need pruning to prevent overfitting. Training requires $O(T \cdot n \log n)$ time on average and $O(n^2)$ in the worst case. Inference needs $O(T \cdot \log n)$ time on average and $O(n)$ in the worst case.

Decision trees possess several characteristics making them more adaptable than alternative classifiers. They accommodate various data formats including discrete and continuous values, with continuous values convertible to categorical ones using thresholds. Furthermore, they manage missing values effectively, which often challenges other classifiers.

Day	Weather	Temperature	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	Mild	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	Strong	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No

Decision trees excel at processing diverse data types without extensive preparation, unlike alternative machine learning approaches requiring data scaling or transformation. Primary focus should involve developing meaningful features that expose significant data relationships, managing missing information carefully, and selecting features judiciously. Excessive irrelevant features may cause trees to learn from noise rather

than genuine patterns, making quality more important than quantity during data preparation.

Three primary strategies exist for managing missing data in decision tree models: removal, substitution, and adaptive learning. Each approach offers distinct benefits and drawbacks depending on missing data characteristics, quantity, and prediction objectives. Removal represents the most straightforward missing data handling method. It involves eliminating rows

or columns containing missing values from datasets. Substitution involves replacing missing values with estimated or alternative values. Various substitution methods exist, including mean, median, mode, or regression-based approaches. Adaptive learning provides the most sophisticated missing data handling approach. It employs machine learning algorithms or techniques to learn from existing data and manage missing information accordingly.

Decision trees encounter scalability difficulties as datasets expand. Primary concerns include extended training periods and increased memory requirements when processing millions of records or thousands of features. As datasets grow, traditional algorithms slow down because they must examine all data repeatedly to identify optimal splits. This can be helped by the use of metrics that determine prediction quality. Accuracy calculates the percentage of correct predictions among all predictions made, providing overall performance understanding. Precision indicates how many positive predictions were actually accurate, which matters when false positives are expensive. Recall or sensitivity, measures how many actual positive cases the model correctly identified, important when missing positive cases is problematic. F1-score merges precision and recall into one metric, offering balanced perspective when considering both false positives and false negatives.

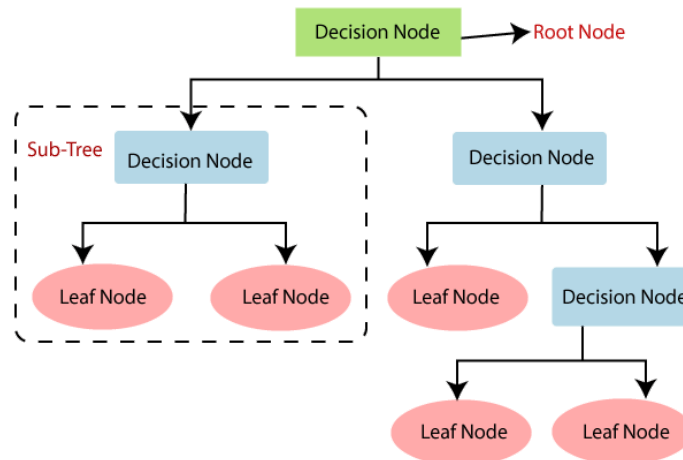
Regularization methods impose constraints on learning algorithms, limiting their flexibility and overfitting potential. For decision trees, regularization occurs through controlling tree depth and complexity. Primary regularization techniques for decision trees include: Maximum Depth which establishes limits on tree growth depth. Minimum Samples Split ensures nodes contain minimum sample quantities before splitting. Minimum Samples Leaf guarantees terminal nodes maintain minimum sample counts.

The Decision Tree Algorithm

A decision tree working starts with a main question known as the root node. This question is derived from the features of the dataset and serves as the starting point for decision-making. From the root node, the tree asks a series of yes/no questions. Each question is designed to split the data into subsets based on specific attributes. For example if the first question is "Is it raining?", the answer will determine which branch of the tree to follow. Depending on the

response to each question you follow different branches. If your answer is "Yes," you might proceed down one path if "No," you will take another path.

This branching continues through a sequence of decisions. As you follow each branch, you get more questions that break the data into smaller groups. This step-by-step process continues until you have no more helpful questions .



In an industrial implementation there are a lot of ways they can implement this, for example, in healthcare, they help doctors diagnose diseases, predict treatment effectiveness, and identify risk factors by evaluating patient symptoms and medical history, in the finance sector uses decision trees to assess credit risk for loans, detect fraudulent transactions, and predict market trends for investment decisions, and in marketing teams leverage these tools to segment customers for targeted campaigns, predict customer churn, and recommend products based on individual preferences and behavior.

Algorithm #2: Support Vector Machine (SVM)

Description

Support Vector Machines (SVM) are a class of supervised learning algorithms used for both classification and regression tasks. They are particularly well-suited for binary classification problems, although extensions such as One-vs-Rest (OvR) and One-vs-One (OvO) allow SVMs to handle multiclass classification as well.

At the heart of an SVM is the idea of finding a decision boundary, known as a hyperplane, that best separates the classes in the training data. Unlike other classifiers that might seek to minimize classification error directly, SVMs aim to maximize the margin—the distance between the hyperplane and the nearest data points from each class, which are referred to as support vectors. A larger margin tends to improve the model's ability to generalize to unseen data.

When the data is not linearly separable, SVMs can use a technique known as the kernel trick. Kernels are mathematical functions that implicitly project the data into a higher-dimensional space where it may become linearly separable, without explicitly computing the transformation. Common kernels include:

- **Linear kernel** – For linearly separable data.
- **Polynomial kernel** – Captures polynomial relationships.
- **Radial Basis Function (RBF) kernel** – Useful for non-linear patterns with a localized decision boundary.
- **Sigmoid kernel** – Similar to activation functions in neural networks.

This kernel-based approach gives SVMs great flexibility, enabling them to model complex decision boundaries while maintaining robust generalization under the right hyperparameter settings. Another key feature of SVMs is the regularization parameter (C), which controls the trade-off between achieving a low training error and a large margin. A small C encourages a wider margin at the cost of some misclassifications (more regularization), while a larger C focuses on correctly classifying all training examples, possibly at the cost of overfitting.

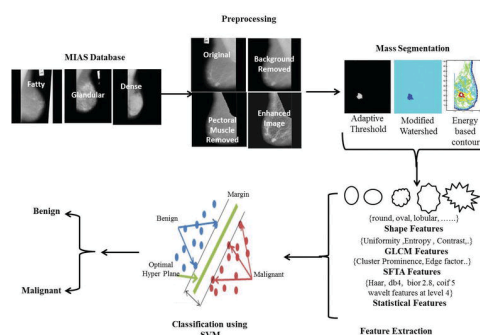
Industrial Implementation

Support Vector Machines are widely used in high-stakes industries, especially where precision and robustness are crucial and datasets are high-dimensional (many features per sample).

Example: Bioinformatics / Medical Diagnosis

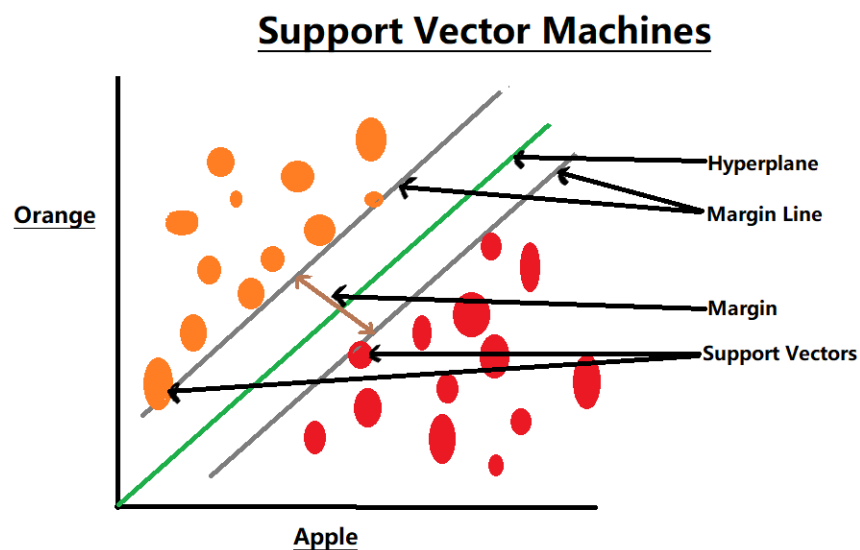
One of the most notable real-world applications of SVMs is in cancer detection, particularly in distinguishing between malignant and benign tumors.

- **Institution:** *National Cancer Institute* and many medical research institutions.
- **Use Case:** Classifying breast cancer tumors using the Wisconsin Diagnostic Breast Cancer (WDBC) dataset, where each sample is described by features like radius, texture, perimeter, and smoothness.
- **Why SVM?:**
 - Performs well with small-to-medium datasets.
 - Excels in high-dimensional spaces, where each instance has dozens or hundreds of features.
 - SVM's ability to handle non-linear relationships using kernels makes it ideal for complex biological data that cannot be separated using linear boundaries.
 - High predictive accuracy and good generalization, which is critical for medical decision-making.



Other Notable Industry Uses:

- **Text Categorization & Spam Detection:** Email providers like Gmail and Outlook use SVMs to classify spam vs. non-spam based on textual features.
- **Financial Sector:** Credit risk assessment and fraud detection, especially when feature spaces are large and complex.
- **Image Recognition:** Early computer vision systems employed SVMs for object recognition tasks before deep learning took over.



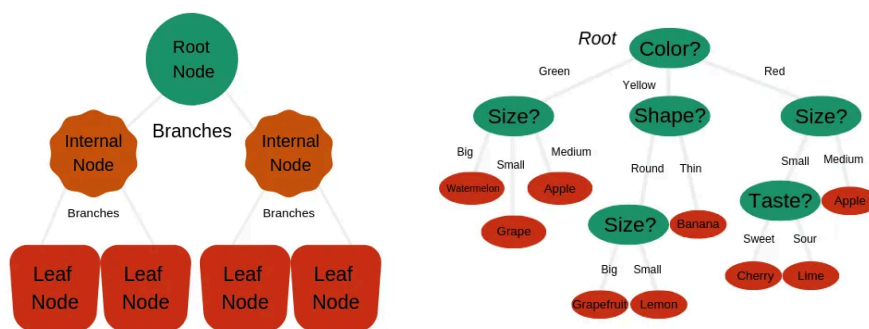
Algorithm #3: Random Forest Algorithm

Description

It is to no surprise we have heard about or have employed decision trees at some point in our lives. Branching out from a simple “what” question, and from there consider the various conditions which may alter the binary (yes or no) answer to the question. Every question makes up the decision nodes in the tree, acting as a means to split the data.

While decision trees are common supervised learning algorithms, they can be prone to problems, such as bias and overfitting. However, when multiple decision trees form an ensemble, they predict more accurate results, particularly when the individual trees are uncorrelated with each other. This structure of data is known in the Machine Learning world as a **Random Forest**.

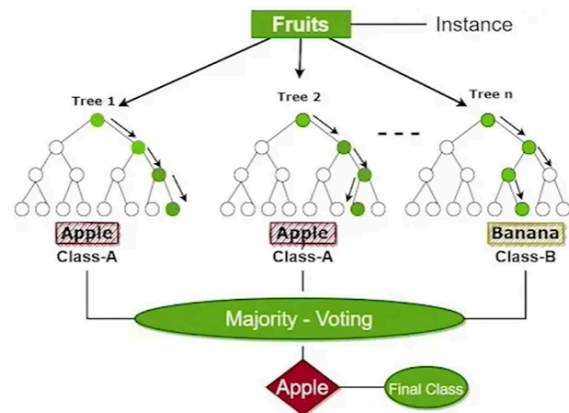
A random forest algorithm works in the same way a decision tree does: it starts with a root node containing a specific data query and from there branches out to possible responses all linked to the same mother node. However, the big difference relies on its structure. Whereas a decision tree consists of only one branch system from a single node, a random forest algorithm is composed of different decision trees, each with the same nodes, but using different data that leads to different leaves. It merges the decisions of multiple decision trees in order to find an answer, which represents the average of all these decision trees.



Random forest algorithms are a **supervised learning** type of ML algorithm, which means the data is derived from a pre-existing dataset with an n-number of independent variables in the

input, and an n-number of dependent variables in the output designed to display the result of a prediction. Such labels in the dataset can be for example, in animals where it depicts the size, fur, ears and weight. The algorithm takes all data received by the input and throws out an estimated result based on the input data. But a random forest would explore many different combinations and sequences of features across trees, preventing the fixed-nature result of the decision tree.

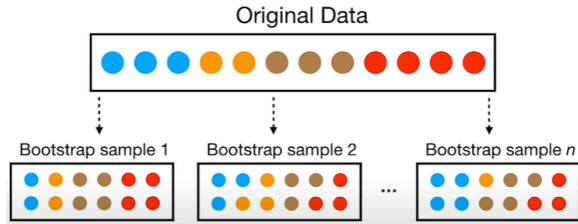
In other words, the random forest algorithm is a **meta-learner**; meaning consisting of many individual learners (trees). The random forest uses multiple random trees classifications to **vote** on an overall classification for the given set of inputs. In general, each individual machine learner vote is given equal weight, this means that each tree votes on the classification of the data by the majority of votes of the different trees, making it useful for **classification** and **regression** purposes.



The way it works is as follows:

1. A data set (commonly referred to as “inbag”) is formed by sampling with replacement members from the training set; this technique is often referred to as “bootstrapping”. The number of examples in the inbag data set is equal to that of the training data set. This new data set may contain duplicate examples from the training set to improve certainty. Using the bootstrapping technique, usually one third of the training set data is not present in the inbag. This leftover data is known as the “out-of-bag data [oob]”.
2. A random number of attributes are chosen for each tree. These attributes form the nodes and leaves using standard tree building algorithms.
3. Each tree is grown to the fullest extent possible without pruning.

Depending on the type of problem, the determination of the prediction will vary. For a regression task, the individual decision trees will be averaged, and for a classification task, a majority vote – the most frequent categorical variable – will yield the predicted class. Finally, the oob sample is then used for cross-validation, finalizing that prediction.



Bagging, on first glance, sounds like it limits the samples of the dataset, but it's useful for two reasons: first, it **enhances accuracy** when random features are used. Second is that **bagging can be used to give ongoing estimates** of the generalization error of the combined ensemble of trees, as well as estimates for the strength and correlation. Generalization error refers to how well a machine learning model performs on new, unseen data. In other words, how well it can generalize from the training data to the real world.

These estimates are done in the OOB sample, which provides leverage to the inbag data to output more accurate results. Assume a training dataset T , from here multiple new datasets T_k are created by sampling with replacement (or bootstrapping). Each T_k is the same size as T but may have repeated examples and may miss some examples from T . Then, a classifier $h(x, T_k)$ is made for each T_k and let them vote to form the bagged predictor. Now focusing the attention to the missing examples of the T_k datasets (x, y) . For each (x, y) , only consider predictions from the classifiers trained without that example. The classifier built from these votes is called the out-of-bag (OOB) classifier for (x, y) . Finally, the OOB classifiers are used to predict the labels for the training examples they didn't see, and the rate percentage of incorrect predictions is the out-of-bag estimate of the generalization error—a measure of how well the model performs on unseen data.

To put it lightly, bagging works by creating many random versions of the training data, building a model on each one, and letting them vote and the majority results form the bagged predictor. This is on one end of the spectrum **for available examples in the bootstrapped datasets**. On the other end of the spectrum, we need a necessary margin of error to improve the accuracy model for those bootstrap datasets which **don't have a particular set of examples**. Derived from these datasets emerges the OOB estimate, the margin of error which increases accuracy for the bootstrap models lacking examples from the general dataset.

Industry Implementations

The Random Forest Algorithm is used to solve both regression and classification problems, making it a versatile model used by engineers worldwide. It is applicable in various sectors such as the following:

- **Finance:** It is a preferred algorithm over others as it reduces time spent on data management and pre-processing tasks. It can be used to evaluate customers with high credit risk, to detect fraud, and option pricing problems.
- **Healthcare:** The random forest algorithm has applications within computational biology, allowing doctors to tackle problems such as gene expression classification, biomarker discovery, and sequence annotation. As a result, doctors can make estimates around drug responses to specific medications.
- **E-commerce:** It can be used for recommendation engines for cross-sell purposes.

As mentioned before, random forest algorithms are best suited for classification and regression tasks. We'll present a case for each one to grasp a better understanding of both cases.

Classification

A common use of the classification method is in client research for companies aiming to learn their target audience, which customers are more likely to buy their products and which don't. This can be done with satisfaction forms, online forms or directly taking surveys with the clients. This will get better closure to classify them as a potential customer or not a potential customer.

Then the data is registered in a dataset ready to be computed, however if the survey is being done with non-numerical values (given the data of the survey was done from quality variables of a store instead of quantifiable variables), the one-hot method must be used. This assigning a number to a categorical variable in order to apply mathematics to the problem.

After the categorical variables are one-hot encoded, the algorithm can get to work. Depending on the results shown it can either entail two conclusions: either most people in the target market are potential customers and the company must further promote their line of product, or they are not potential customers and therefore, the company must re-evaluate their line of product for their target audience.

Label Encoding

Product Purchased	Categorical #	Purchase Price
Toilet Paper	1	\$4.00
Shampoo	2	\$6.50
Conditioner	3	\$8.50

One Hot Encoding

Toilet Paper	Shampoo	Cleanser	Price
1	0	0	\$4.00
0	1	0	\$6.50
0	0	1	\$8.50

Regression

Since the regression model calculates estimates, a great area of practice is in tax income or household income data registry. Depending on the peoples opinions with their financial income (no matter if its job, household, financial, etc), an estimate will be generated for each person. Each person represents a tree, and each question of the survey is a node which eventually culminates in leaf nodes.

After capturing the data of each individual (tree), the next phase is categorizing the data in the algorithm. Each tree will output a result and use a random forest to find an average income between all of the decision trees. The result will be an accurate estimate of the average income – whether it be job, household or financial – of the people surveyed.

CONCLUSIONS

Conclusion Decision Tree

Decision trees are algorithms that function like flowcharts, starting from the root and branching with the data that we give them. Their main strength lies in their ability to handle both classification and regression tasks, process different data types without extensive preparation, and effectively manage missing values. Despite the limitation that this algorithm has, that can be there scalability challenges or the complexity being hard when in worst-case scenarios, the algorithm is straightforward and practical for the sector to use it, making it easy to use.

Conclusion Support Vector Machines (SVM)

Support Vector Machines (SVM) offer a powerful and versatile approach to both classification and regression problems, especially in scenarios with high-dimensional and complex data. By maximizing the margin between classes and leveraging kernel functions, SVMs achieve strong generalization and high accuracy, even with non-linear relationships. Their ability to handle small to medium-sized datasets with many features makes them particularly valuable in fields such as medical diagnostics, text categorization, and financial risk analysis. While SVMs require careful tuning of parameters, their robustness and precision make them a reliable choice for critical real-world applications.

Conclusion Random Forest Algorithms

Overall, random forest algorithms cover a large area of decision - making algorithms which entail vast data samples which may be too much for only one decision tree. The methods applied like the bagging samples and out-of-bag estimates improve on the algorithm's efficiency to arrive at more accurate results and estimates by adding a margin of error with a determined number of examples in the dataset, an efficient and time-saving way to troubleshoot data.

It is like asking a panel of diverse experts, each with slightly different perspectives and then going with the consensus—which usually leads to a better decision than narrowing it down to the result of just one. There is less result bias and the dataset is a lot more extensive.

The structure is user-friendly, it is highly flexible with any type of data and their accuracy aims to reduce data bias, making their practical applications wide-spread in many industrial sectors across the world.

General Conclusions

In conclusion, Decision Trees, Support Vector Machines (SVM), and Random Forests each offer unique strengths and trade-offs when applied to machine learning tasks. Decision Trees are highly interpretable and simple to implement but can suffer from overfitting in complex datasets. Support Vector Machines provide strong performance, especially in high-dimensional spaces, but require careful tuning of parameters and are computationally intensive. Random Forests, as an ensemble method, strike a balance by combining the simplicity of decision trees with improved accuracy and robustness through bagging and random feature selection.

Each algorithm excels in different industrial applications depending on the context: Decision Trees are often used in financial and business rule systems, SVMs in image recognition and bioinformatics, and Random Forests in fraud detection and healthcare analytics. Ultimately, the best choice of algorithm depends on the specific dataset characteristics, computational resources, and the need for interpretability versus predictive power. Understanding these distinctions is key to making informed decisions in real-world machine learning implementations.

Bibliography

- [Decision Trees in Machine Learning: Two Types \(+ Examples\) | Coursera](#)
- [1.10. Decision Trees — scikit-learn 1.6.1 documentation](#)
- [9 Decision Tree – Interpretable Machine Learning.](#)
- [What is a Decision Tree? | IBM.](#)
- [What feature engineering is necessary with tree based algorithms? - Data Science Stack Exchange](#)
- [How to Build a Decision Tree for Machine Learning](#)
- <https://www.linkedin.com/advice/0/how-can-you-handle-missing-data-decision-tree-agv6f?lang=en>
- [Why We Need to Do Regularization in Decision Tree Machine Learning? | by Deryl Baharudin Sopandi | Medium](#)
- [Decision Tree Algorithm](#)
- [Understanding Decision Tree Algorithm in Machine Learning - Shiksha Online](#)
- Burges, C. J. C. (1998). A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*, 2(2), 121–167. <https://doi.org/10.1023/A:1009715923555>
- Hsu, C.-W., Chang, C.-C., & Lin, C.-J. (2010). *A practical guide to support vector classification* (Technical report). Department of Computer Science, National Taiwan University. <https://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf>
- Furey, T. S., Cristianini, N., Duffy, N., Bednarski, D. W., Schummer, M., & Haussler, D. (2000). Support vector machine classification and validation of cancer tissue samples using microarray expression data. *Bioinformatics*, 16(10), 906–914. <https://doi.org/10.1093/bioinformatics/16.10.906>
- -----
- Livingston, F. (2005). Random Forests [Frederick Livingston]. Data Jobs Test. [https://datajobstest.com/data-science-repo/Random-Forest-\[Frederick-Livingston\].pdf](https://datajobstest.com/data-science-repo/Random-Forest-[Frederick-Livingston].pdf)
- Breiman, L. (2001, octubre). Random Forests. Springer. <https://link.springer.com/article/10.1023/a:1010933404324>
- Schott, M. (2021, 9 diciembre). Random Forest Algorithm for Machine Learning - Capital One Tech - Medium. Medium. <https://medium.com/capital-one-tech/random-forest-algorithm-for-machine-learning-c4b2c8cc9feb>
- IBM. (2025, 17 abril). Random Forest. What is a random forest? <https://www.ibm.com/think/topics/random-forest>
- Bootstrap Aggregation, Random Forests and Boosted Trees | QuantStart. (s. f.). <https://www.quantstart.com/articles/bootstrap-aggregation-random-forests-and-boosted-trees/>
- Infinite Codes. (2024, 17 septiembre). All Machine Learning algorithms explained in 17 min [Video]. YouTube. <https://www.youtube.com/watch?v=E0Hmnixke2g>